

Exercise 04

Ternary Operator and Recursion

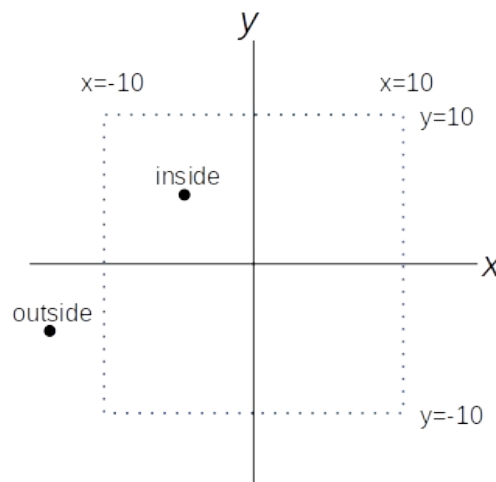
In this exercise, you'll get some practice using the ternary operator. This kind of operator shows up in a lot of languages (including Java), so getting comfortable with it will help you out even if you're not working in C.

You'll find a partial implementation, and some sample inputs and expected outputs on the course homepage. You can download them from there, or, if you're logged in on `remote.eos.ncsu.edu`, you can copy them to your current directory using the following shell command.

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise04/* .
```

Counting Points

The job of the `square.c` program is to read 2D point coordinates from standard input and count the number of points that are inside a 20 x 20 square centered on the origin. Each point is given on an input line as two values of type `double`, representing `x` and `y` coordinates. We'll say a point is inside the square if the absolute value of its coordinates are both less than or equal to 10.



Completing the Implementation

You'll need to fill in the bodies of three functions to finish the program. To help us get practice using the ternary operator, we'll say you can't use any `if` statements or `switch` statements. Instead, you'll implement any branching behavior you need with the ternary operator. Likewise, you're not permitted to use any looping statements (no `for` loops, no `while` loops, no `do-while` loops). Instead, you'll implement looping using the ternary operator along with recursion.

The job of the **`absolute()`** function is to return the absolute value of its argument. If the argument is already positive, it can just return it unmodified. If the argument is negative, it will return the negated value of the argument. This should be a straightforward application of the ternary operator.

The **`inside()`** function takes the `x` and `y` coordinates of a point as parameters. Its job is to return value 1 if the point is inside the 20 x 20 square and return zero if it's not. It can use the `absolute()` function to help simplify testing for containment in the square.

The `countPoints()` function is a little more interesting. It will read all the points from the input and return the total number that are inside the square. The `main()` function already has code to print out this number after `countPoints()` returns. The `countPoints()` function will be implemented recursively. On each call, it will attempt to read a point from input. If it can't read coordinates for a point, it will return a count of zero. This is the base case of the recursion. Remember that the return value from `scanf()` will tell you whether it successfully got the input you asked it to read.

If `countPoints()` can successfully read a point, p , it will check to see if p is inside the square. If it is, it will recursively count the rest of the points and return that count plus 1 (plus 1 because its point, p , is one more point inside the square). If p isn't inside the square, it will recursively count the rest of the points and then return that count plus zero.

The example to the right shows how this can work on the first sample input. The first call to `countPoints()` reads a point that's inside the square. It adds 1 for this point and calls itself recursively to count the rest of the points.

countPoints() 1 +

Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

The next recursive call reads a point that's not inside the square, so it adds in a zero and calls itself recursively to count the rest of the points.

countPoints() 1 +
 ↓
 countPoints() 0 +

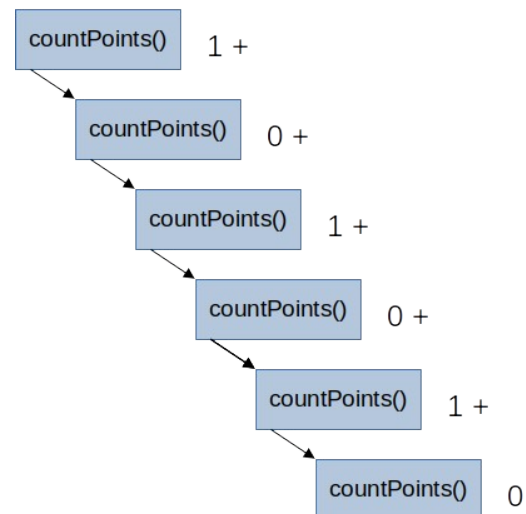
Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

After calling `countPoints()` five levels deep, we've read all the points. For any point that's in the square, `countPoints()` will add in a 1 to the count. Otherwise, it will add in a zero. The ones and zeros in each call will actually get added up on the way out of the recursion.

countPoints() 1 +
 ↓
 countPoints() 0 +
 ↓
 countPoints() 1 +
 ↓
 countPoints() 0 +
 ↓
 countPoints() 1 +

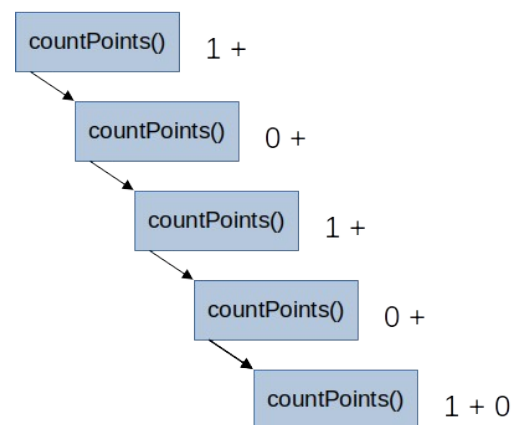
Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

The first sample input just has five points, so the sixth call to `countPoints()` can't read any more points. It returns a zero immediately.



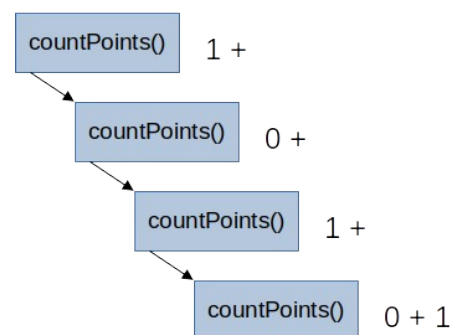
Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

When the this zero is returned to the caller, it can add in the one for its point and return.



Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

Up one level in the recursion, we have a point that's not in the square, so the caller adds a zero and returns.



Input-1.txt	
8.49	8.95
10.21	-3.17
-4.94	8.05
-9.07	12.35
-6.67	1.62

This continues all the way up the chain of recursive calls, each caller getting the count returned by `countPoints()` and adding either a one or a zero depending on whether its point is in the 20 x 20 square or not. The topmost call is able to return the total count back to main.

`countPoints()` 1 + 2

Input-1.txt

```
8.49 8.95
10.21 -3.17
-4.94 8.05
-9.07 12.35
-6.67 1.62
```

Sample Execution

Once your program is ready, you should be able to run it as follows. Here, we're running with input redirected from sample input 1 and letting the output go to the terminal.

```
$ ./square < input-1.txt
3
```

To make sure your program's output is exactly right, you will want to check it against the expected output. Below, we're running the program with input read from `input-1.txt` and capturing output to a file named `output.txt`. Immediately after execution, we check to make sure the program terminated successfully, then we compare its output to the expected output. Looks like everything worked.

```
$ ./square < input-1.txt > output.txt
$ echo $?
0
$ diff output.txt expected-1.txt
```

Similar commands will let us check the program against another test case:

```
$ ./square < input-2.txt > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

Submitting your Solution

When you're done, check your program to make sure you don't use any looping statements (no for loops, no while loops. ... no do/while loops), no if statements. You should be able to do all your looping and conditional logic using the ternary operator and recursion. Submit your completed **square.c** file to the exercise_04 assignment in Moodle.